pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Consumer Credit

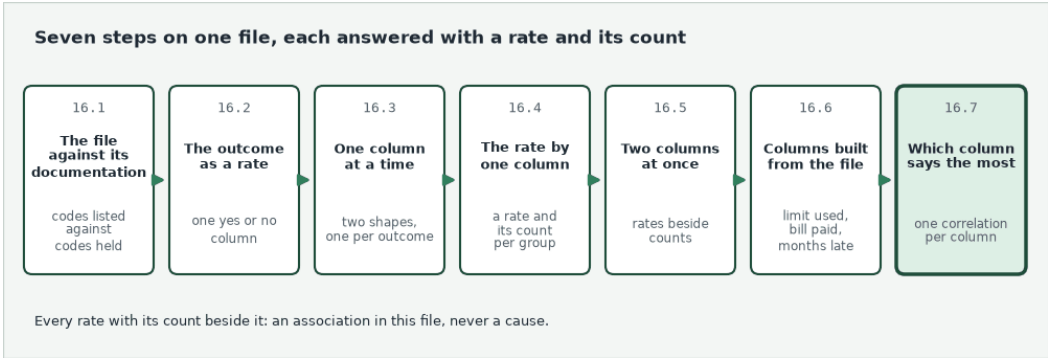
WEEK 4, DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note, and a public file

We cover consumer lenders, and everyone quotes a public file of 30,000 card accounts. Before we do: does it match its documentation, and what share defaulted the next month? Which groups defaulted more, on how many accounts? Does that change with two groupings at once, or with columns we build: limit used, bill paid, months late? And which column says the most? Rates and counts in this file only: no causes, no lending advice.



Real public data about people, not synthetic. The page describes "customers' default payments in Taiwan", from "April to September, 2005". It names neither the lender nor how the accounts were chosen, so nothing here carries to another market.

Title	Default of Credit Card Clients
Creator	I-Cheng Yeh
Source	UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
DOI	https://doi.org/10.24432/C55S3H
License	Creative Commons Attribution 4.0 International (CC BY 4.0), https://creativecommons.org/licenses/by/4.0/legalcode
Citation	Yeh, I. (2009). Default of Credit Card Clients [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55S3H.
Changed for this course	The header row X1 to Y dropped; saved as credit_card_default.csv.

16.1 The file against its documentation

```
raw = pd.read_csv(data_folder + 'credit_card_default.csv')
print(raw.shape)
raw.head(3)
(30000, 25)
```

One row is one card account; the first, aged 24, defaulted the next month.

```
print(raw.dtypes.value_counts())
print('Missing:', raw.isna().sum().sum())
print('IDs unique:', raw['ID'].is_unique, '| first:', raw['ID'].min(), '| last:', raw['ID'].max())
print('Repeated rows:', raw.duplicated().sum(),
      '| without ID:', raw.drop(columns='ID').duplicated().sum())
int64    25
Name: count, dtype: int64
Missing: 0
IDs unique: True | first: 1 | last: 30000
Repeated rows: 0 | without ID: 35
```

35 rows match another on everything but the ID; two accounts can hold the same values, so they stay.

Nothing is missing, which is not the same as every value explained.

```
raw.default
AttributeError: 'DataFrame' object has no attribute 'default'
```

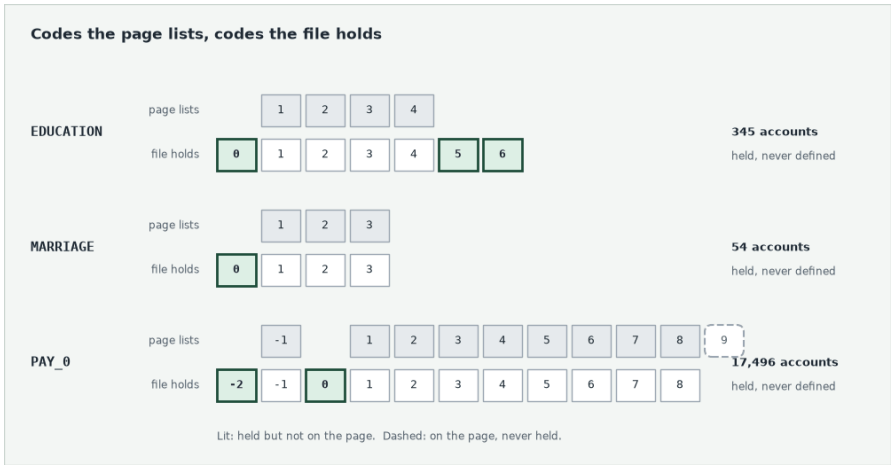
No: the column is "default payment next month", and a name after a dot holds no space, so only brackets reach it.

```
cards = raw.copy()
cards = cards.rename(columns={'default payment next month': 'default'})
cards['default'].value_counts()
default
0    23364
1     6636
Name: count, dtype: int64
```

Decision 1: the outcome is "default" in "cards"; the file on disk is unchanged.

A difference in rates is an association, not a cause. First, the file against its documentation.

Codes listed, codes held: label them, keep them



More than half the accounts hold -2 or 0, codes not on the page, in every month.

Decision 3: -2 and 0 keep their own numbers; a month is a delay only at a code of 1 or more.

```
print(cards['SEX'].unique())
print(cards['EDUCATION'].value_counts().sort_index())
print(cards['MARRIAGE'].value_counts().sort_index())
print('Undocumented education:', (~cards['EDUCATION'].isin([1, 2, 3, 4])).sum())
Undocumented education: 345
```

"EDUCATION" also holds 0, 5, and 6, on 345 accounts, and "MARRIAGE" holds 0 on 54: codes the page never defines.

```
cards['sex'] = cards['SEX'].map({1: 'male', 2: 'female'})
cards['education'] = cards['EDUCATION'].map({1: 'graduate school', 2: 'university',
                                             3: 'high school', 4: 'others'})
cards['marriage'] = cards['MARRIAGE'].map({1: 'married', 2: 'single', 3: 'others'})
print(cards[['sex', 'education', 'marriage']].isna().sum())
cards['education'] = cards['education'].fillna('not documented')
cards['marriage'] = cards['marriage'].fillna('not documented')
sex            0
education    345
marriage      54
dtype: int64
```

Decision 2: the undocumented codes stay, labelled "not documented".

Excel side by side. "EDUCATION" in D, the outcome in Y, accounts in rows 2 to 30001.

Job	pandas	Excel	Result
Undocumented codes	`~isin([1, 2, 3, 4])`	=COUNTIF(D2:D30001, "<1")+COUNTIF(D2:D30001, ">4")	345
A label	`map(), fillna()`	=IFERROR(VLOOKUP(D2, codes, 2, FALSE), "not documented")	#N/A is the gap map() leaves

16.2, 16.3 A rate needs its count; the limit by outcome

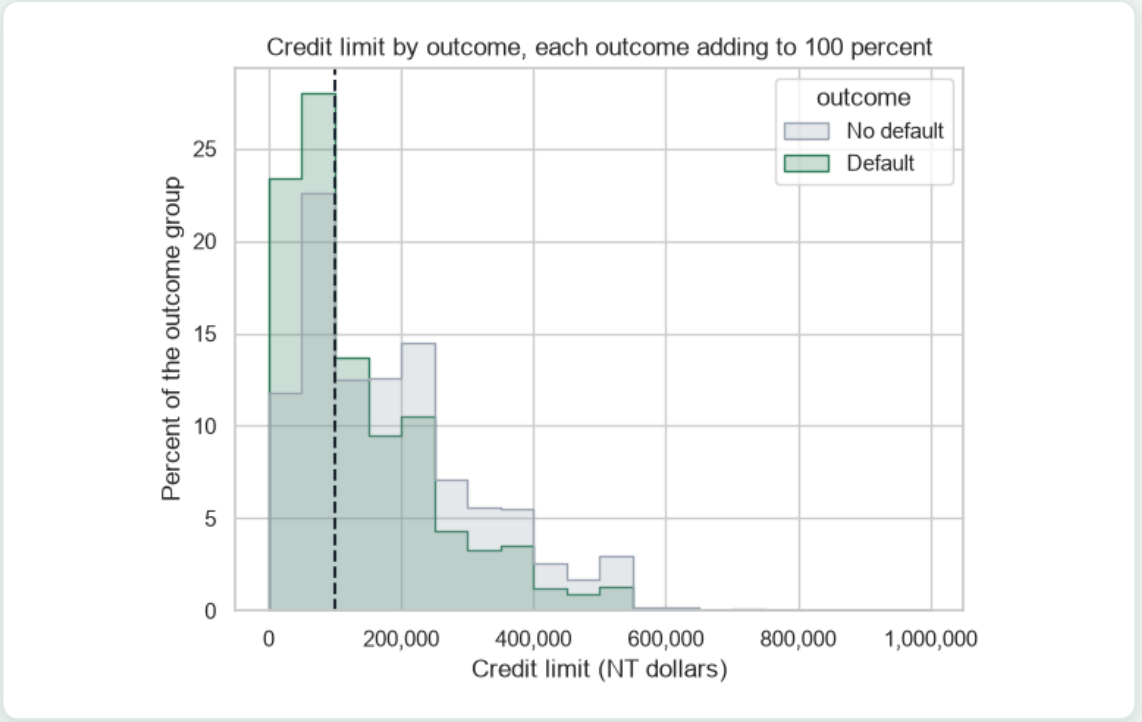
```
code_zero = cards[cards['EDUCATION'] == 0]
print(len(code_zero), 'accounts,', code_zero['default'].sum(), 'defaults')
print(round(1 / 14 * 100, 2), round(1 / 30000 * 100, 3))
14 accounts, 0 defaults
7.14 0.003
```

None of the 14 defaulted; one default would make the rate 7.14 percent, and move the whole file by 0.003 points.

No rate without its count.

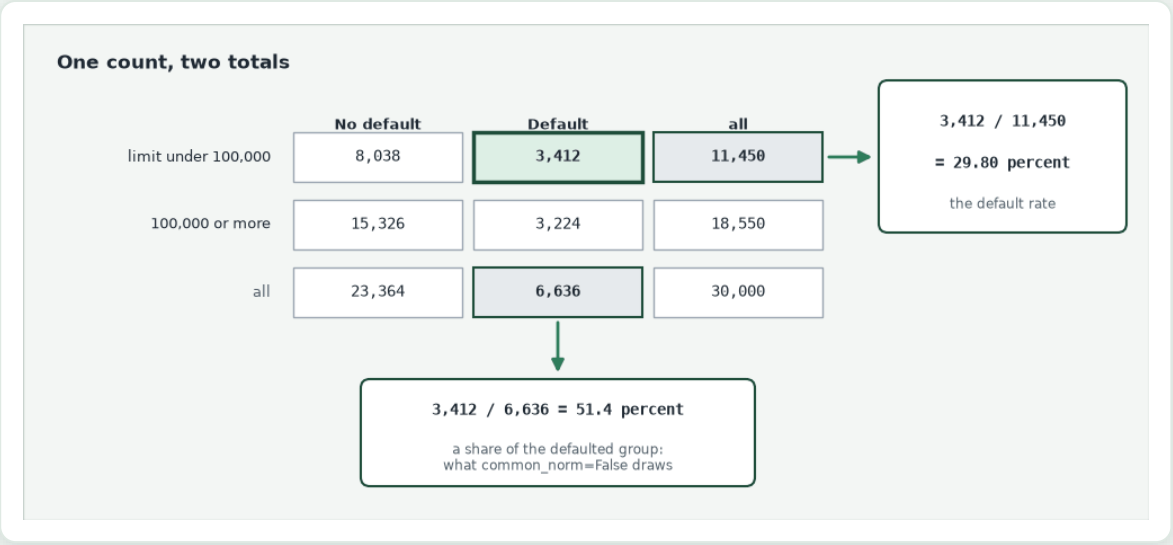
```
print(cards['LIMIT_BAL'].describe().round(0))
print('Skew:', round(cards['LIMIT_BAL'].skew(), 2), '| limits:', cards['LIMIT_BAL'].nunique())
print('At exactly 50,000:', (cards['LIMIT_BAL'] == 50000).sum())
count      30000.0
mean      167484.0
std       129748.0
min        10000.0
25%        50000.0
50%       140000.0
75%       240000.0
max      1000000.0
Name: LIMIT_BAL, dtype: float64
Skew: 0.99 | limits: 81
At exactly 50,000: 3365
```

The mean sits above the median, a tail to the right, and 3,365 accounts sit at exactly 50,000.



Left of the dashed line, the green Default bars stand taller than the grey.

A share of a group, bills, and values far from the rest



A share of a group is not a default rate: say which total a percentage is of.

```
print(cards[['BILL_AMT1', 'PAY_AMT1']].describe().round(0))
print(cards[['BILL_AMT1', 'PAY_AMT1']].skew().round(2))
bill = cards['BILL_AMT1']
print((bill < 0).sum(), (bill == 0).sum(), (bill > cards['LIMIT_BAL']).sum(),
      (cards['PAY_AMT1'] == 0).sum())
```

Both skew right, so quote the medians, 22,382 and 2,100. The page explains neither the 590 bills below zero nor the 2,008 at zero, and all of them stay.

```
for column in ['LIMIT_BAL', 'BILL_AMT1', 'PAY_AMT1']:
    q1, q3 = cards[column].quantile(0.25), cards[column].quantile(0.75)
    far = (cards[column] < q1 - 1.5 * (q3 - q1)) | (cards[column] > q3 + 1.5 * (q3 - q1))
    print(column, far.sum(), 'far:', round(cards.loc[far, 'default'].mean() * 100, 2),
          'percent | the rest:', round(cards.loc[~far, 'default'].mean() * 100, 2))
print('Defaults among the far payments:', cards.loc[far, 'default'].sum())
LIMIT_BAL 167 far: 11.38 percent | the rest: 22.18
BILL_AMT1 2400 far: 19.88 percent | the rest: 22.32
PAY_AMT1 2745 far: 12.28 percent | the rest: 23.11
Defaults among the far payments: 337
```

They default less, in all three: 12.28 percent past the payment's fence against 23.11. Deleting those 2,745 "outliers" would delete 337 defaults.

Excel side by side. "LIMIT_BAL" in B.

Job	pandas	Excel	Result
Share of a group	`common_norm=False`	=COUNTIFS(Y2:Y30001, 1, B2:B30001, "<100000")/COUNTIF(Y2:Y30001, 1)	0.514

Decision 5: every far value stays.

16.4 One rate table, with its count and margin

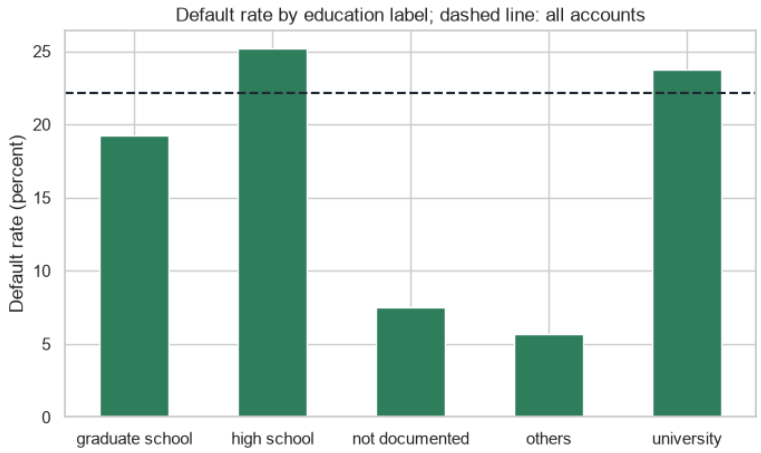
Sex, education, marital status, and age describe people: these rates explain nothing and are no basis for treating people differently.

```
def rate_table(column):
    table = cards.groupby(column)['default'].agg(accounts='count', defaults='sum', rate_pct='mean')
    p = table['rate_pct']
    table['margin_pts'] = (1.96 * np.sqrt(p * (1 - p) / table['accounts']) * 100).round(2)
    table['rate_pct'] = (p * 100).round(2)
    return table
```

A rough margin of error says how far a rate on "n" accounts could move by chance: 1.96 x sqrt(p x (1 - p) / n), with "p" the rate as a fraction. It assumes a random draw, which the page never claims.

education	accounts	defaults	rate_pct	margin_pts
graduate school	10,585	2,036	19.23	0.75
high school	4,917	1,237	25.16	1.21
not documented	345	26	7.54	2.79
others	123	7	5.69	4.09
university	14,030	3,330	23.73	0.70

The count sits under the square root, so four times the accounts halves the margin.



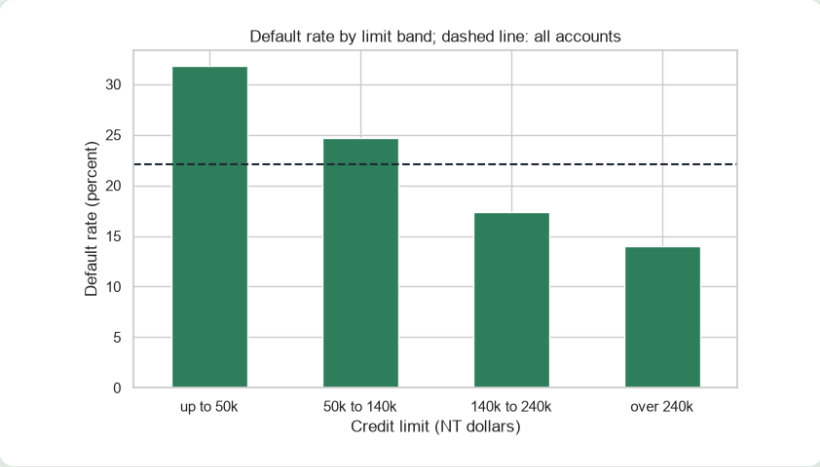
"others" defaulted at 5.69 percent on 123 accounts, give or take 4.09. Code 0 alone, 0 of 14, would get a margin of 0.00: the formula failing, not certainty. Every bar is as wide on 123 accounts as on 14,030: the count lives in the table.

Bands: age, credit limit, and the payment

```
cards['age_band'] = pd.cut(cards['AGE'], bins=[20, 30, 40, 50, 60, 80],
                           labels=['21 to 30', '31 to 40', '41 to 50', '51 to 60', '61 to 79'])
rate_table('age_band')
```

age_band	accounts	defaults	rate_pct	margin_pts
21 to 30	11013	2471	22.44	0.78
31 to 40	10713	2189	20.43	0.76
41 to 50	6005	1399	23.30	1.07
51 to 60	1997	504	25.24	1.91
61 to 79	272	73	26.84	5.27

Age dips from 22.44 to 20.43, then climbs to 26.84 on the 272 oldest.



The rate falls from 31.79 percent in the lowest band to 13.98 in the highest, each on over 7,000 accounts.

```
pd.qcut(cards['PAY_AMT1'], 10)
ValueError: Bin edges must be unique:
```

5,249 zero payments, over a tenth of the file, make the first two edges both 0. "duplicates='drop'" would merge them in silence and change the question.

```
cards['payment_band'] = pd.cut(cards['PAY_AMT1'], bins=[-1, 0, 2000, 5000, 20000, np.inf],
                               labels=['nothing', '1 to 2,000', '2,001 to 5,000',
                                       '5,001 to 20,000', 'over 20,000'])
rate_table('payment_band')
```

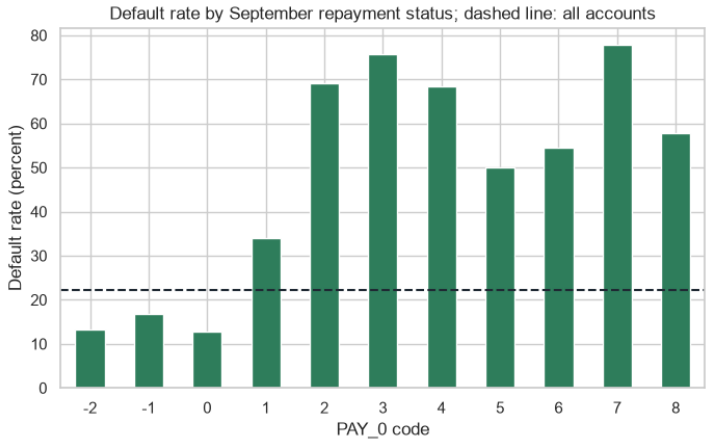
payment_band	accounts	defaults	rate_pct	margin_pts
nothing	5249	1887	35.95	1.30
1 to 2,000	9166	2085	22.75	0.86
2,001 to 5,000	8021	1624	20.25	0.88
5,001 to 20,000	6245	915	14.65	0.88
over 20,000	1319	125	9.48	1.58

Those that paid nothing defaulted at 35.95 percent, those that paid over 20,000 at 9.48.

Repayment status separates the most

```
status_rates = rate_table('PAY_0')
print(status_rates)
       accounts  defaults  rate_pct  margin_pts
PAY_0
-2         2759       365     13.23        1.26
-1         5686       954     16.78        0.97
 0        14737      1888     12.81        0.54
 1         3688      1252     33.95        1.53
 2         2667      1844     69.14        1.75
 3          322       244     75.78        4.68
 4           76        52     68.42       10.45
 5           26        13     50.00       19.22
 6           11         6     54.55       29.43
 7            9         7     77.78       27.16
 8           19        11     57.89       22.20
```

Code 2 defaulted at 69.14 percent; the undocumented -2 and 0 hold the two lowest rates, 13.23 and 12.81. Codes 4 to 8 rest on 141 accounts, with margins up to 29.43 points.



Repayment status separates the most so far: 12.81 to 69.14 percent.

Excel side by side. "LIMIT_BAL" in B, "SEX" in C, "AGE" in F.

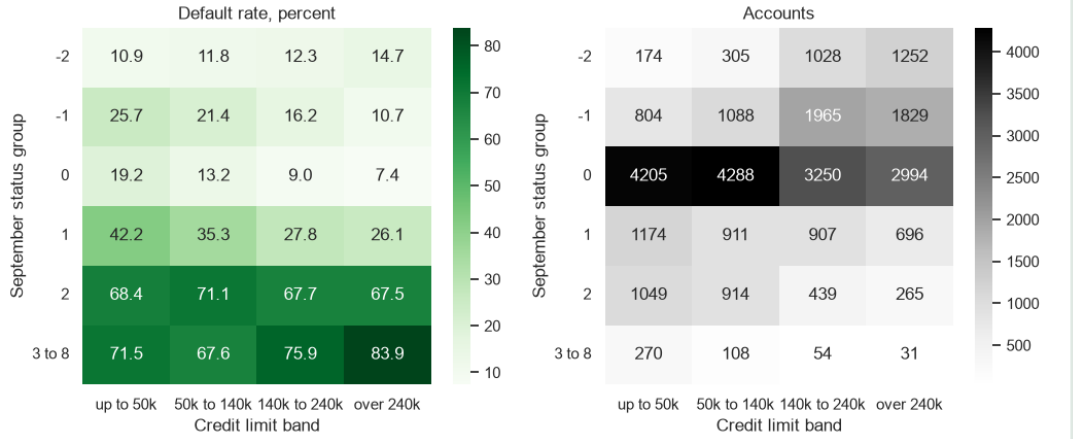
Job	pandas	Excel	Result
Defaults	`sum`	=SUMIF(C2:C30001, 1, Y2:Y30001)	2,873
Bar range	`vmin=0, vmax=100`	Data Bars, More Rules, Minimum 0, Maximum 100	fixed bars
Age band	`pd.cut()`	=VLOOKUP(F2, bands, 2, TRUE), on a table of lower edges	a label
Quartile edge	`pd.qcut()`	=PERCENTILE.INC(B2:B30001, 0.25)	50,000

16.5 Rates beside counts

```
cards['status_group'] = pd.cut(cards['PAY_0'], bins=[-3, -2, -1, 0, 1, 2, 8],
                               labels=['-2', '-1', '0', '1', '2', '3 to 8'])
rate_pivot = (pd.pivot_table(cards, values='default', index='status_group', columns='limit_band',
                             aggfunc='mean') * 100).round(1)
count_pivot = pd.pivot_table(cards, values='default', index='status_group', columns='limit_band',
                             aggfunc='count')
print('3 to 8:', (cards['status_group'] == '3 to 8').sum(),
      '| accounts in the table:', count_pivot.sum().sum())
3 to 8: 463 | accounts in the table: 30000
```

```
p = rate_pivot / 100
margin_pivot = (1.96 * np.sqrt(p * (1 - p) / count_pivot) * 100).round(1)
print('31 accounts:', margin_pivot.loc['3 to 8', 'over 240k'], 'points')
print('4,288 accounts:', margin_pivot.loc['0', '50k to 140k'], 'points')
31 accounts: 12.9 points
4,288 accounts: 1.0 points
```

The 83.9 percent could be 12.9 points out; a cell of 4,288 accounts, 1.0.



The darkest rate, 83.9 percent, sits on one of the palest counts: 31 accounts. Code 0 falls from 19.2 to 7.4 percent as the limit rises; code 2 stays between 67.5 and 71.1; code -2 climbs from 10.9 to 14.7.

Excel side by side. "LIMIT_BAL" in B, "PAY_0" in G.

Job	pandas	Excel	Result
A cell's count	`count_pivot`	=COUNTIFS(G2:G30001, 2, B2:B30001, "<=50000")	1,049
Its rate	`rate_pivot`	=AVERAGEIFS(Y2:Y30001, G2:G30001, 2, B2:B30001, "<=50000")	0.684

Read every rate with its cell's count.

16.6 Limit used, and the share of the bill paid

```
cards['utilisation'] = cards['BILL_AMT1'] / cards['LIMIT_BAL']
cards['utilisation_band'] = pd.cut(cards['utilisation'],
                                   bins=[-np.inf, 0, 0.25, 0.5, 0.75, 1, np.inf],
                                   labels=['0 or below', 'to 0.25', 'to 0.5', 'to 0.75', 'to 1',
                                           'over 1'])
rate_table('utilisation_band')
```

utilisation_band	accounts	defaults	rate_pct	margin_pts
0 or below	2598	643	24.75	1.66
to 0.25	11440	1923	16.81	0.69
to 0.5	3588	780	21.74	1.35
to 0.75	3579	929	25.96	1.44
to 1	6680	1725	25.82	1.05
over 1	2115	636	30.07	1.95

The median bill uses 0.314 of its limit, and 2,115 bills exceed it; no limit is zero, so no division fails. The rate climbs from 16.81 percent on 11,440 accounts to 30.07 on those over their limit.

```
cards['share_paid'] = np.where(cards['BILL_AMT1'] > 0,
                               cards['PAY_AMT1'] / cards['BILL_AMT1'], np.nan)
```

The 27,402 shares have a median of 0.061 against a mean of 1.776, so quote the median.

```
paid_rates = rate_table('paid_band')
print(paid_rates)
print('Accounts in the table:', paid_rates['accounts'].sum())
             accounts  defaults  rate_pct  margin_pts
paid_band
under 0.05      11723      2989     25.50        0.79
0.05 to 0.1      5063      1107     21.86        1.14
0.1 to 0.5       5176      1084     20.94        1.11
0.5 to 1         1569       213     13.58        1.69
1 or more        3871       600     15.50        1.14
Accounts in the table: 27402
```

The rate falls from 25.50 percent under 0.05 of the bill to 13.58 at 0.5 to 1, then edges up to 15.50. The table holds 27,402: a gap has no band, so "groupby" leaves it out.

Excel side by side

With the six codes in G to L, "=COUNTIF(G2:L2, ">=1")" counts the delays.

Three built columns; the months with a delay climb the steepest.

Predict first: decide, then run each in the notebook

Predict 1

"default" holds 23,364 zeros and 6,636 ones. What does "cards['default'].mean()" return? A) 6636 B) 0.2212 C) An error: a yes or no has no mean.

```
print(cards['default'].mean())
print(cards['default'].sum() / len(cards))
overall_rate_pct = round(cards['default'].mean() * 100, 2)
print('Default rate:', overall_rate_pct, 'percent')
```

Predict 2

2,873 accounts labelled male defaulted, and 3,763 labelled female. Which label had the higher default rate? A) female B) male C) These two numbers cannot say.

```
print(cards.groupby('sex')['default'].agg(['count', 'sum', 'mean']))
```

Predict 3

"pd.qcut(cards['LIMIT_BAL'], 4)" cuts at the quartiles, so each band should hold a quarter of 30,000. What does "value_counts()" show? A) 7,500 in each band B) The first band holds more than 7,500. C) An error

```
pd.qcut(cards['LIMIT_BAL'], 4).value_counts().sort_index()
```

Predict first: zero bills, months late, and a correlation

Predict 4

2,008 September bills are zero. What does "(cards['PAY_AMT1'] / cards['BILL_AMT1']).mean()" return? A) A "ZeroDivisionError" B) A number between 0 and 1 C) "inf"

```
share_paid = cards['PAY_AMT1'] / cards['BILL_AMT1']
print('Mean:', share_paid.mean())
print('Infinite:', np.isinf(share_paid).sum(), '| missing:', share_paid.isna().sum())
```

Predict 5

Account 2's six codes, September back to April, are -1, 2, 0, 0, 0, 2. How many months does "(cards[pay_columns] >= 1).sum(axis=1)" count for it? Write a number.

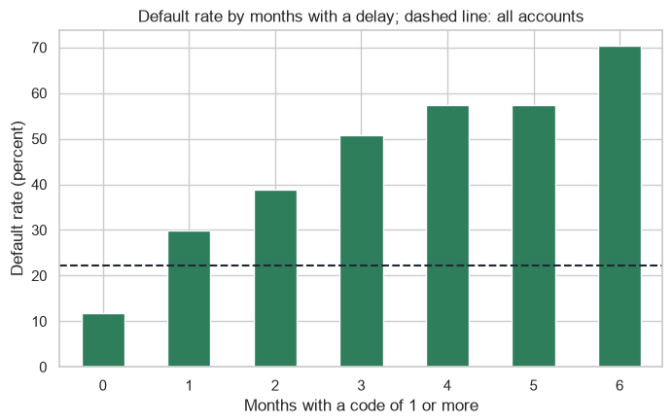
```
cards['months_delayed'] = (cards[pay_columns] >= 1).sum(axis=1)
print('Account 2:', cards.loc[1, 'months_delayed'])
rate_table('months_delayed')
```

Predict 6

The limit bands fell from 31.79 to 13.98 percent. The correlation of "LIMIT_BAL" with "default" is closest to: A) -0.9 B) -0.15 C) 0

```
columns = ['months_delayed', 'PAY_0', 'LIMIT_BAL', 'AGE', 'default']
cards[columns].corr()['default'].round(3)
```

16.7 Months with a delay, and the note answered



Months with a delay: 11.71 percent with none, 70.32 with six. The note is answered.

Story

The file came with a 2009 paper by I-Cheng Yeh and Che-hui Lien in Expert Systems with Applications that compared ways to predict card defaults (https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients). These 30,000 accounts return in Course 4: today describes, Course 4 predicts.

The note asked	Answer
Documentation	No gaps; undefined codes kept.
Groups	Status code 0, 12.81 percent; code 2, 69.14.
Two groupings	83.9 percent, on 31 accounts.
Built columns	11.71 to 70.32 percent by months delayed.

Be proud: every rate you wrote stops where the file stops.

Next, in Analyzing Text, Part 1, the data stops being numbers: published statements become a table you can check.